

Golf/logica.c

```

1  #include <stdlib.h>
2  #include <math.h>
3  #include "Jogo.h"
4
5  //
6  //
7  // Lógica de jogo
8  //
9  //
10
11 // Função que dermina se uma jogada é válida (apenas se o módulo da
    diferença entre os valores for 1)
12 int jogadaValida (carta c, carta d) {
13     if (abs (c.num - d.num) == 1) } se forem cartas consecutivas, a
14         return 1;                  jogada é válida
15     else
16         return 0;
17 }
18
19 // Função que cria um baralho com 52 cartas
20 void criarBaralho(jogo *j) {
21     int i = 0; Array começa em 0
22     for (int naipes = 0; naipes < 4; naipes++) { ← Para cada naipe
23         for (int num = 1; num <= 13; num++) { ← Cria as 13 cartas
24             (*j).baralho[i].num = num;
25             (*j).baralho[i].naipes = naipes;
26             i++; Guarda as naipes posições do baralho
27         }
28     }
29     (*j).cartas_baralho = 52; ← define que o baralho tem 52 cartas
30 }
31
32 // Função que baralha as cartas usando o método Fisher-Yates
33 void baralharCartas(jogo *j) { Começa na ultima carta
34     for (int i = 51; i > 0; i--) {
35         Se gera um valor aleatório entre 0 e i-1
36         int r = rand() % (i + 1); ← gera um valor entre 0 e a posição atual
37         carta temp = (*j).baralho[i];
38         (*j).baralho[i] = (*j).baralho[r];
39         (*j).baralho[r] = temp; ← Troca a carta em i com a carta em r
40     }
41 }
42
43 // Função que distribui as cartas pelas 7 pilhas
44 void distribuirCartas(jogo *j) {
45     for (int p = 0; p < 7; p++) { ← Para 7 pilhas
46         for (int c = 0; c < 5; c++) { ← Repete 5 vezes
47             (*j).cartas_baralho--; ← tira 1 ao m de cartas no baralho

```


Cria um inteiro que vai ser usado como índice, faz começa em 51

```

47     int topo_do_baralho = (*j).cartas_baralho;
48     (*j).pilhas[p][c] = (*j).baralho[topo_do_baralho];
49 }
50 (*j).quantidade_pilhas[p] = 5; Colocamos na pilha (p) e carta (c)
51 }
52 }
53
54 // Função que retira a carta do topo do baralho e coloca-a no descarte
para iniciar o jogo
55 void descartar(jogo *j){
56     (*j).cartas_descarte = 0; ← Diz que as cartas no descarte são 0
57     (*j).cartas_baralho--; ← tira um do n° de cartas do baralho
58     int topo_baralho = (*j).cartas_baralho; ← tirando como o índice da carta
em cima
59
60     int topo_descarte = (*j).cartas_descarte; ← Quando o índice onde
61     (*j).descarte[topo_descarte] = (*j).baralho[topo_baralho]; ← vai pra a carta
62     → Faz a troca da carta e depois atualiza o número de cartas
63     (*j).cartas_descarte++;
64 }
65
66 // Função que invoca outras funções necessárias para o começo do jogo
67 void começarJogo(jogo *j) {
68     criarBaralho(j);
69     baralharCartas(j);
70     distribuirCartas(j);
71     descartar(j);
72 }
73
74 // Função que faz o biscoar de uma carta; devolve se este foi bem sucedido
ou não
75 int biscoarCarta(jogo *j) {
76     if ((*j).cartas_baralho > 0) { ← Apenas permite biscoar se o n° cartas for
77         (*j).cartas_baralho--; ← retiramos uma do baralho
78         int topo_baralho = (*j).cartas_baralho; ← É o número do índice da
79         int topo_descarte = (*j).cartas_descarte; ← carta que vamos tirar
80
81         (*j).descarte[topo_descarte] = (*j).baralho[topo_baralho];
82         (*j).cartas_descarte++;
83         return 1;
84     }
85     return 0;
86 }
87
88 // Função que verifica a validade da jogada e, se for válida, move a carta
da pilha para o descarte, retornando o estado da operação
89 int moverDaPilha(jogo *j, int opcao) {
90     int p = opcao - 1; → move 1 a 6 mas o array vai de 0 a 6
91     if ((*j).quantidade_pilhas[p] > 0) { ← apenas se tiver cartas
92         int topo_p = (*j).quantidade_pilhas[p] - 1; → Pega na quantidade
93         int topo_d = (*j).cartas_descarte - 1; → de cartas e tira 1
para descolar o índice

```



```

94  carta c_pilha = (*j).pilhas[p][topo_p];
95  carta c_descarte = (*j).descarte[topo_d];
96
97  if (jogadaValida(c_pilha, c_descarte) == 1) {
98      (*j).descarte[topo_d + 1] = c_pilha;
99      (*j).cartas_descarte++;
100     (*j).quantidade_pilhas[p]--;
101     return 1;
102 }
103 return 0;
104 }
105 return -1;
106 }
107
108 // Função que verifica a vitória
109 int verificarVitoria(jogo *j) {
110     int pilhas_vazias = 0;
111     for (int i = 0; i < 7; i++) {
112         if ((*j).quantidade_pilhas[i] == 0) {
113             pilhas_vazias++;
114         }
115     }
116     if (pilhas_vazias == 7) {
117         return 1;
118     }
119     return 0;
120 }

```

Vai buscar as cartas e guarda em var. temp.

Se a jogada for válida

copia a carta do topo da pilha para o próximo espaço vazio do descarte

Usamos os 3 para retirar as cartas e imprimir mensagens de sucesso pois nunca seguimos erros (0 e -1) só em casos de sucesso

0 → jogada ilegal

-1 → jogada feita com pilha vazia

Se a pilha tiver vazia, aumenta pilhas_vazias

Se todas as pilhas estiverem vazias retorna 1